FOR COMPLETE BEGINNERS

How does code run?

Complete Modern Java, episode 1: your first Java program, explained

> You write instructions as source code. Tools check them and run them. A program can run and still be wrong.

Sarah buys two loaves of bread and one carton of milk: three items, $3.99. The same receipt program, in Java and in TypeScript.

Change milk to the word "one". Java refuses to compile. TypeScript reports the same mistake, but by default still writes a file that runs and prints "Items: 2one" and "Total: $NaN".

In this episode

- Source code
- Compiler and runtime
- The terminal and the IDE
- javac, java, tsc, node

Coming next

- Installing Java and IntelliJ (episode 2)
- Installing Node and TypeScript (episode 3)
- Errors and the debugger (episode 4)

At the time of recording: Java 25 LTS, Node 24 LTS, TypeScript 7.

01 What each run printed

JAVA	`Items: 3, Total: $3.99`
TYPESCRIPT	`Items: 3, Total: $3.99`
JAVA, milk = "one"	`javac: String cannot be converted to int; no .class written`
TYPESCRIPT, milk = "one"	`tsc: TS2322, .js still written; node prints Items: 2one, Total: $NaN`
BOTH, milk = 2	`Items: 4, Total: $4.98`

milk = 1 unless stated.

02 Java: compile, then run

A Java 25 compact source file: void main() and nothing around it. javac checks the code and writes GroceryReceipt.class, full of bytecode. java starts the JVM, which runs it. java GroceryReceipt.java does both steps at once.

java/ep01/GroceryReceipt.java

```
void main() {
    int bread = 2;
    int milk = 1;
    double breadPrice = 1.50;
    double milkPrice = 0.99;

    double total = bread * breadPrice + milk * milkPrice;

    IO.println("Sarah's groceries");
    IO.println("Items: " + (bread + milk));
    IO.println("Total: $" + total);
}
```

terminal

```
javac GroceryReceipt.java
java GroceryReceipt
```

> Items: 3, Total: $3.99
>
> Run on a real machine, 2026-10-18: Java 25, Node 24.22.0, TypeScript 7.0.2.

03 TypeScript: check, write JavaScript, run

tsc checks the types and writes groceryReceipt.js with the types removed. node runs the JavaScript. Node can also run the .ts file directly: it removes the types and does not check them.

typescript/ep01/groceryReceipt.ts

```
const bread: number = 2;
const milk: number = 1;
const breadPrice: number = 1.50;
const milkPrice: number = 0.99;

const total: number = bread * breadPrice + milk * milkPrice;

console.log("Sarah's groceries");
console.log("Items: " + (bread + milk));
console.log("Total: $" + total);
```

terminal

```
npx tsc groceryReceipt.ts
node groceryReceipt.js
```

> Items: 3, Total: $3.99
>
> Run on a real machine, 2026-10-18: Java 25, Node 24.22.0, TypeScript 7.0.2.

04 Both found the mistake. One setup still let it run.

Checking the code, producing a file and running it are three separate steps. Java will not produce the file. tsc, by default, reports the error and still writes it; --noEmitOnError stops that. A number plus text joins them as text (2 + "one" is "2one"), and text times a price is NaN, not a number.

the change

```
int milk = "one";            // Java
const milk: number = "one";  // TypeScript
```

> javac: incompatible types: String cannot be converted to int. tsc: TS2322, then node printed Items: 2one, Total: $NaN
>
> Run on a real machine, 2026-10-18: Java 25, Node 24.22.0, TypeScript 7.0.2.

Check yourself

1. What does javac write, and what runs it?
2. What does tsc write, and what runs it?
3. Why did TypeScript still produce a file?
4. Why "2one", and why NaN?
5. If milk goes from 1 to 2, what prints? (Items: 4, Total: $4.98)

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code, and the commands for every output: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

JEP 512: Compact Source Files and Instance Main Methods
https://openjdk.org/jeps/512

Node.js: running TypeScript natively
https://nodejs.org/api/typescript.html

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-18 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.